

Linear Algebra Analyzer

A Comprehensive C++ Implementation

Covering: RREF · LU/QR/SVD Decompositions · Four Fundamental Subspaces
Eigenvalues & Eigenvectors · Jordan Form · Projections · Positive Definiteness
Similar Matrices · Change of Basis · $Ax = b$ Solver

Sarvesh S

IIT Madras — `ed23b042@smail.iitm.ac.in`

GitHub: `algorithmalchemist69/linear-algebra-cpp`

Theory basis: Gilbert Strang, *Introduction to Linear Algebra* (5th ed.) and *Linear Algebra and Its Applications*.

Numerical methods are explicitly identified throughout this document where they go beyond what Strang's textbook covers.

June 18, 2026

0 Contents

1	Project Overview	2
2	Strang’s “Big Picture” of Linear Algebra	2
3	Implemented Operations — Theory and Implementation	2
3.1	Row Reduction and Reduced Row Echelon Form (RREF)	2
3.2	LU Decomposition with Partial Pivoting	3
3.3	QR Decomposition via Modified Gram-Schmidt	4
3.4	Singular Value Decomposition (SVD)	4
3.5	The Four Fundamental Subspaces	5
3.6	Orthonormal Bases via Gram-Schmidt	5
3.7	Projection Matrices	5
3.8	Solving $Ax = b$ — Least Squares via Pseudoinverse	6
3.9	Determinant	6
3.10	Eigenvalues and Eigenvectors	6
3.10.1	Symmetric matrices — QR Iteration	6
3.10.2	General (non-symmetric) matrices — Faddeev-LeVerrier + Durand-Kerner	7
3.11	Positive Definiteness	8
3.12	Jordan Form and Number of Jordan Blocks	8
3.13	Similar Matrices and Change of Basis	9
3.14	Inverse and Pseudoinverse	9
4	Summary of Numerical Methods	10
5	Convergence and Precision	10
6	Worked Example	11
7	Limitations and Future Work	11
8	References	11

1 Project Overview

This project is a self-contained C++ (C++17) command-line tool that performs complete linear algebra analysis on an arbitrary $m \times n$ real matrix A entered by the user. It implements every major topic from Gilbert Strang’s linear algebra curriculum plus several classical numerical algorithms required to make those topics computationally feasible.

1.0 File structure

File	Size	Purpose
<code>linalg.h</code>	~120 lines	Class declaration and result-type structs
<code>linalg.cpp</code>	~450 lines	Full implementation of all algorithms
<code>main.cpp</code>	~200 lines	Interactive CLI, formatted output
<code>Makefile</code>	8 lines	Build with <code>g++ -std=c++17 -O2</code>

1.0 Build and run

```
cd linear_algebra
make           # compiles all three sources into ./linalg
./linalg      # enter dimensions and matrix interactively
```

2 Strang’s “Big Picture” of Linear Algebra

The conceptual backbone of the project is Strang’s *Big Picture*: every matrix $A \in \mathbb{R}^{m \times n}$ of rank r divides both $\mathbb{R}^n$ and $\mathbb{R}^m$ into two orthogonal pairs of subspaces.

The Four Fundamental Subspaces (Strang Ch. 3)

$$\begin{aligned}
 \mathbb{R}^n &= \underbrace{\mathcal{C}(A^T)}_{\text{row space, dim } r} \perp \underbrace{\mathcal{N}(A)}_{\text{null space, dim } n-r} \\
 \mathbb{R}^m &= \underbrace{\mathcal{C}(A)}_{\text{column space, dim } r} \perp \underbrace{\mathcal{N}(A^T)}_{\text{left null space, dim } m-r}
 \end{aligned}$$

This decomposition is central to understanding solvability ($Ax = b$ has a solution iff $b \in \mathcal{C}(A)$), the structure of solutions (particular solution plus any element of $\mathcal{N}(A)$), and the geometry of projections. **The project computes all four subspaces and their orthonormal bases.**

3 Implemented Operations — Theory and Implementation

3.1 Row Reduction and Reduced Row Echelon Form (RREF)

Strang reference: Ch. 2 (Elimination), Ch. 3 (Vector Spaces).

Given $A \in \mathbb{R}^{m \times n}$, Gauss-Jordan elimination transforms A into its unique RREF R . The algorithm finds pivot positions, establishes rank r , identifies free variables, and directly produces a basis for all four fundamental subspaces.

Algorithm: Gauss-Jordan (RREF)

Input: $A \in \mathbb{R}^{m \times n}$

Output: RREF R , list of pivot columns, rank r

Steps:

1. For each column j (left to right), among rows $\geq$ current pivot row, find the row with the largest absolute entry (*partial pivoting*). Swap it to the current pivot row.
2. Divide the pivot row by its pivot entry (make pivot = 1).
3. Eliminate *all* other rows (both above and below) in column j .
4. Record j as a pivot column; advance to next pivot row.

Numerical method note: Partial pivoting (choosing the largest absolute entry as pivot) is a standard numerical stabilization technique, *not* part of Strang's theoretical treatment but essential in practice to prevent division by near-zero numbers and amplification of floating-point errors.

Outputs extracted from RREF:

- $\text{rank}(A) = r = \text{number of pivot columns}$.
- **Pivot positions:** pairs (i_k, j_k) where the k -th leading 1 sits.
- **Null space basis:** for each free column f , the vector $\mathbf{x}_f$ with $x_f = 1$, $x_{f'} = 0$ for other free columns, and $x_{p_k} = -R_{k,f}$ for each pivot row k .
- **Row space basis:** the nonzero rows of R .
- **Column space basis:** the pivot columns of the *original* A .
- **Left null space basis:** null space of A^T (computed by applying the same procedure to A^T).

3.2 LU Decomposition with Partial Pivoting

Strang reference: Ch. 2 ($A = LU$).

For a square matrix A , we factor $PA = LU$ where P is a permutation matrix (recording row swaps), L is unit lower triangular, and U is upper triangular.

Why LU?

Gaussian elimination *is* LU factorization. Strang shows that the multipliers $\ell_{ij} = a_{ij}/a_{jj}$ (at the time of elimination) fill the lower triangle of L . The upper triangle of U is what remains after elimination.

Partial pivoting: at step k , the row with the largest $|a_{ik}|$ ($i \geq k$) is swapped to position k and recorded in P . This guarantees $|L_{ij}| \leq 1$ for all $i > j$, bounding error growth.

Use in the project: LU is used to compute:

- $\det(A) = (-1)^s \prod_i U_{ii}$, where $s = \text{number of row swaps}$.
- Forward/back substitution for $Ax = b$ when A is square and nonsingular.
- Display: L , U , P are printed separately.

3.3 QR Decomposition via Modified Gram-Schmidt

Strang reference: Ch. 4 (Orthogonality), Gram-Schmidt.

For $A \in \mathbb{R}^{m \times n}$ with $m \geq n$, we write $A = QR$ where $Q \in \mathbb{R}^{m \times n}$ has orthonormal columns and $R \in \mathbb{R}^{n \times n}$ is upper triangular.

Algorithm: Modified Gram-Schmidt (MGS)

Input: A with columns $\mathbf{a}_1, \dots, \mathbf{a}_n$

Output: Q (orthonormal columns), R (upper triangular)

Steps: for $j = 1, \dots, n$:

1. $R_{jj} \leftarrow \|\mathbf{u}_j\|$ (norm of current working column). If $R_{jj} < \varepsilon$, skip (linearly dependent column).
2. $\mathbf{q}_j \leftarrow \mathbf{u}_j / R_{jj}$.
3. For $k > j$: $R_{jk} \leftarrow \mathbf{q}_j^T \mathbf{u}_k$; $\mathbf{u}_k \leftarrow \mathbf{u}_k - R_{jk} \mathbf{q}_j$ (project out the new $\mathbf{q}_j$ *immediately*).

Classical vs Modified Gram-Schmidt: Classical GS subtracts projections onto *all* previously found $\mathbf{q}$ vectors at the end. Modified GS subtracts each projection *as it is found*. Both give the same result in exact arithmetic, but MGS is significantly more numerically stable: the loss of orthogonality in CGS is $O(\varepsilon \kappa^2)$ while MGS achieves $O(\varepsilon \kappa)$, where $\kappa = \|A\| \|A^{-1}\|$ is the condition number. This is a *numerical methods* distinction beyond Strang's theoretical treatment.

Uses in the project: QR decomposition is displayed to the user and also serves as the workhorse inside the eigenvalue QR iteration (Sec. 3.9).

3.4 Singular Value Decomposition (SVD)

Strang reference: Ch. 7 ($A = U\Sigma V^T$, the pinnacle of linear algebra).

The SVD writes any $A \in \mathbb{R}^{m \times n}$ as

$$A = U\Sigma V^T$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal and $\Sigma \in \mathbb{R}^{m \times n}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0 = \sigma_{r+1} = \dots$. Strang calls this “the most important factorization in linear algebra.”

SVD and the Four Subspaces (Strang Ch. 7)

- **Columns of V :** orthonormal basis for $\mathbb{R}^n$ split into row space $(v_1, \dots, v_r)$ and null space $(v_{r+1}, \dots, v_n)$.
- **Columns of U :** orthonormal basis for $\mathbb{R}^m$ split into column space $(u_1, \dots, u_r)$ and left null space $(u_{r+1}, \dots, u_m)$.
- $Av_i = \sigma_i u_i$ for $i = 1, \dots, r$; $Av_i = 0$ for $i > r$.

Numerical method used — SVD via eigendecomposition of $A^T A$:

The project computes SVD using the following steps, which are a *numerical methods choice* (not an algorithm Strang derives in detail):

1. Form the symmetric PSD matrix $B = A^T A \in \mathbb{R}^{n \times n}$.
2. Compute the eigendecomposition of B (using the QR iteration, Sec. 3.9): $B = V\Lambda V^T$, with $\lambda_i = \sigma_i^2$.

3. Sort eigenvalues descending; form $\sigma_i = \sqrt{\lambda_i}$.
4. For each nonzero σ_i : $\mathbf{u}_i = A\mathbf{v}_i/\sigma_i$.
5. Complete U with an orthonormal basis for $\mathcal{N}(A^T)$ via iterated Gram-Schmidt against the existing $\mathbf{u}_i$.

Limitation: The numerically preferred algorithm is Golub-Reinsch bidiagonalization (used in LAPACK’s `dgesvd`). The $A^T A$ approach can lose relative accuracy for small singular values because squaring the matrix squares the condition number. For the pedagogical matrices (size ≤ 6) typical in a course, this is entirely sufficient.

3.5 The Four Fundamental Subspaces

Strang reference: Ch. 3.5 (“Dimensions of the Four Subspaces”).

All four subspaces are extracted from the RREF computation:

Subspace	Notation	Dim	How computed
Null space	$\mathcal{N}(A)$	$n - r$	Free-variable vectors from RREF
Column space	$\mathcal{C}(A)$	r	Pivot columns of original A
Row space	$\mathcal{C}(A^T)$	r	Nonzero rows of RREF, stored as columns
Left null space	$\mathcal{N}(A^T)$	$m - r$	Null space of A^T

Rank-Nullity Theorem: $\text{rank}(A) + \text{nullity}(A) = n$ (columns), confirmed numerically for every matrix entered.

3.6 Orthonormal Bases via Gram-Schmidt

Strang emphasizes that orthonormal bases are “the best” bases for computation (Chapters 4 and 7). The project applies the Gram-Schmidt process to the basis vectors of each of the four subspaces to produce orthonormal bases:

$$\text{Orth}\mathcal{N}(A), \quad \text{Orth}\mathcal{C}(A), \quad \text{Orth}\mathcal{C}(A^T), \quad \text{Orth}\mathcal{N}(A^T).$$

The implementation uses Modified Gram-Schmidt (Sec. 3.3), skipping any column whose projection onto the existing orthonormal set falls below $\varepsilon = 10^{-9}$ (numerically linearly dependent).

3.7 Projection Matrices

Strang reference: Ch. 4.2 (Projections onto subspaces).

Projection Formula (Strang)

If A has full column rank, the projection onto $\mathcal{C}(A)$ is

$$P = A(A^T A)^{-1} A^T.$$

For the general (possibly rank-deficient) case, replace $(A^T A)^{-1} A^T$ with the Moore-Penrose pseudoinverse A^+ :

$$P_{\mathcal{C}(A)} = AA^+, \quad P_{\mathcal{N}(A)} = I - P_{\mathcal{C}(A)}.$$

The project computes three projection matrices:

- P_{col} : onto column space (via pseudoinverse, handles all ranks).
- $P_{\text{null}} = I - P_{\text{col}}$: onto null space.
- P_{row} : onto row space (computed as $P_{\text{col}}(A^T)$ applied to A^T).

These satisfy $P^2 = P$ (idempotent) and $P^T = P$ (symmetric).

3.8 Solving $Ax = b$ — Least Squares via Pseudoinverse

Strang reference: Ch. 4.3 (Least squares), Ch. 7.4 (Pseudoinverse).

Given $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$, the project finds the minimum-norm least-squares solution:

$$\hat{x} = A^+b = V\Sigma^+U^Tb$$

where Σ^+ replaces each nonzero σ_i with $1/\sigma_i$.

Three Cases (Strang Big Picture)

- $b \in \mathcal{C}(A)$: exact solution exists; $\hat{x}$ is the one with $\hat{x} \in \mathcal{C}(A^T)$ (minimum norm).
- $b \notin \mathcal{C}(A)$: no exact solution; $\hat{x}$ minimizes $\|Ax - b\|_2$ (projection of b onto $\mathcal{C}(A)$).
- A full rank, square: $\hat{x} = A^{-1}b$ (unique solution).

The residual $\|A\hat{x} - b\|_2$ is printed; if it is $> 10^{-6}$, the program notes that $b \notin \mathcal{C}(A)$ and $\hat{x}$ is only a least-squares solution.

3.9 Determinant

Strang reference: Ch. 5 (Determinants).

The determinant is computed via Gaussian elimination (without RREF — only forward elimination to U):

$$\det(A) = (-1)^s \prod_{i=1}^n U_{ii}$$

where s is the number of row swaps performed during partial pivoting. Strang's three properties of the determinant (anti-symmetry under row exchange, multilinearity, $\det(I) = 1$) all hold for this formula. If any $U_{ii} = 0$ (within tolerance ε), $\det(A) = 0$ is returned immediately.

3.10 Eigenvalues and Eigenvectors

Strang reference: Ch. 6.

The project uses **two different algorithms** depending on whether A is symmetric.

3.10.1 Symmetric matrices — QR Iteration

Spectral Theorem (Strang Ch. 6.4)

Every real symmetric matrix has real eigenvalues and an orthonormal eigenvector basis: $A = Q\Lambda Q^T$.

Algorithm: Unshifted QR Iteration (symmetric case)**Input:** symmetric $A \in \mathbb{R}^{n \times n}$ **Output:** eigenvalues (diagonal of T), eigenvectors (columns of V)**Initialise:** $T \leftarrow A, V \leftarrow I$.**Iterate:** while $\|\text{off-diagonal entries of } T\|_F > \varepsilon n$:

1. Factor $T = QR$ (Modified Gram-Schmidt).
2. $T \leftarrow RQ$ (reverse the order).
3. $V \leftarrow VQ$ (accumulate orthogonal factors).

Return: eigenvalues = T_{ii} ; eigenvectors = columns of V .

Why it works: each step performs a similarity transformation $T \leftarrow Q^T T Q = RQ$ (since $T = QR \Rightarrow RQ = Q^{-1} T Q$), so eigenvalues are preserved. For symmetric A , the off-diagonal Frobenius norm decreases monotonically (Strang Ch. 6 discusses diagonalization; the convergence proof uses the fact that all eigenvalues are real and the QR step is a similarity). Convergence rate is $O(|\lambda_i / \lambda_{i+1}|^k)$ per iteration, which is fast when eigenvalues are well-separated.

Why unshifted (not Rayleigh-quotient shifted): A shifted variant $T - \mu I = QR$, $T \leftarrow RQ + \mu I$ accelerates convergence dramatically, but when the shift μ equals an eigenvalue *exactly* (e.g., a matrix with a repeated eigenvalue of 2), the shifted $T - \mu I$ becomes singular and the QR decomposition fails — a column has norm zero and no valid unit vector can be formed. The unshifted iteration avoids this completely, at the cost of slower convergence (still < 200 iterations for typical 3×3 matrices).

3.10.2 General (non-symmetric) matrices — Faddeev-LeVerrier + Durand-Kerner

Non-symmetric matrices may have complex eigenvalues (e.g., a cyclic permutation matrix has eigenvalues that are cube roots of unity). Pure QR iteration on such matrices can cycle without converging (a cyclic permutation's QR factorization gives $Q = A, R = I$, so $RQ = A$ — no change at all). Two classical numerical algorithms are combined to handle this correctly.

Step 1 — Faddeev-LeVerrier Algorithm (characteristic polynomial).

Algorithm: Faddeev-LeVerrier**Input:** $A \in \mathbb{R}^{n \times n}$. **Initialise:** $B \leftarrow I_n$.**Output:** coefficients $c_0, \dots, c_{n-1}$ of $p(\lambda) = \lambda^n + c_{n-1}\lambda^{n-1} + \dots + c_0$ ($= \det(\lambda I - A)$).**For** $k = 1, 2, \dots, n$:

1. $B \leftarrow AB$
2. $c_{n-k} \leftarrow -\text{tr}(B) / k$
3. If $k < n$: $B \leftarrow B + c_{n-k} I$

This gives the exact characteristic polynomial via n matrix multiplications. By the Cayley-Hamilton theorem, $p(A) = 0$. The coefficients have direct algebraic meaning: $c_{n-1} = -\text{tr}(A)$, $c_0 = (-1)^n \det(A)$, and the intermediate c_k are sums of $k \times k$ principal minors (Strang Ch. 5 discusses minors; Faddeev-LeVerrier is a **numerical method** beyond his scope).

Step 2 — Durand-Kerner simultaneous root-finding.**Algorithm: Durand-Kerner (Weierstrass method)****Input:** degree- n polynomial p . **Output:** all roots $z_1, \dots, z_n \in \mathbb{C}$.**Init:** place z_k on a circle of radius $r_0 = 1 + \max_j |c_j/c_n|$ (Cauchy bound): $z_k \leftarrow r_0 e^{2\pi i k / n + 0.17i}$.**Iterate** until $\max_i |z_i^{\text{new}} - z_i| < 10^{-12}$:

$$z_i \leftarrow z_i - \frac{p(z_i)}{\prod_{j \neq i} (z_i - z_j)} \quad \forall i = 1, \dots, n$$

Why this works: the correction term is the Newton step for the deflated polynomial $p(z)/\prod_{j \neq i} (z - z_j)$, whose root near z_i is exactly z_i itself. All n roots are updated *simultaneously*, giving global quadratic convergence for simple roots. This algorithm naturally finds complex roots — the iterates z_k live in $\mathbb{C}$ from the start — making it ideal for general matrices.

Numerical considerations:

- *Repeated roots:* Durand-Kerner converges linearly (not quadratically) at multiple roots. Small spurious imaginary parts ($|\text{Im}(z_i)| \sim 10^{-5}$) arise. The project classifies an eigenvalue as “real” when $|\text{Im}(z_i)| < 10^{-4} \max(1, |\text{Re}(z_i)|)$, handling this gracefully.
- *Eigenvalue grouping in Jordan analysis:* near-equal roots are clustered with relative tolerance 10^{-3} to count algebraic multiplicity.

Eigenvectors. For each real eigenvalue λ_k , the eigenvector is found by computing $\mathcal{N}(A - \lambda_k I)$ via RREF — which is exactly Strang’s method of “solving $(A - \lambda I)\mathbf{x} = 0$.” For complex eigenvalues, eigenvectors require complex arithmetic and are not computed (the Jordan form analysis uses only multiplicities).

3.11 Positive Definiteness

Strang reference: Ch. 6.5 (“Positive Definite Matrices”).

Strang’s Tests for Positive Definiteness

A symmetric matrix A is positive definite iff *any one* of these hold: (a) all eigenvalues $\lambda_i > 0$; (b) all pivots > 0 ; (c) all upper-left determinants > 0 ; (d) $\mathbf{x}^T A \mathbf{x} > 0$ for all $\mathbf{x} \neq 0$.

The project uses criterion (a): calls the eigenvalue solver and checks all real eigenvalues are $> \varepsilon$. Similarly, A is positive semi-definite iff all eigenvalues are $\geq -\varepsilon$.

3.12 Jordan Form and Number of Jordan Blocks

Strang reference: Ch. 6 (briefly; Jordan form is the general structure when eigenvectors do not span $\mathbb{R}^n$).

Jordan Decomposition

Every square matrix A is similar to its Jordan form $J = P^{-1}AP$, where J is block-diagonal with Jordan blocks of the form

$$J_k(\lambda) = \begin{pmatrix} \lambda & 1 & & \\ & \lambda & \ddots & \\ & & \ddots & 1 \\ & & & \lambda \end{pmatrix}.$$

Key quantities for each eigenvalue λ :

- **Algebraic multiplicity** m_a : how many times λ appears as a root of $p(\lambda) = \det(\lambda I - A)$.
- **Geometric multiplicity** $m_g = \dim \mathcal{N}(A - \lambda I) = n - \text{rank}(A - \lambda I)$: how many linearly independent eigenvectors correspond to λ .
- **Number of Jordan blocks** for $\lambda = m_g$.
- A is diagonalizable iff $m_a = m_g$ for every eigenvalue.

Jordan block sizes: the m_g blocks for eigenvalue λ have sizes that sum to m_a . The project distributes them as evenly as possible (first block gets any extra 1 if m_a is not divisible by m_g). The geometric multiplicity is computed directly from $\text{rank}(A - \lambda I)$ via RREF.

3.13 Similar Matrices and Change of Basis

Strang reference: Ch. 6.2 (Diagonalization $A = S\Lambda S^{-1}$), Ch. 7 (Change of basis).

Two matrices $A, B \in \mathbb{R}^{n \times n}$ are *similar* ($A \sim B$) if $B = P^{-1}AP$ for some invertible P . Similar matrices represent the same linear transformation in different bases; they have the same eigenvalues, the same determinant, the same trace, and the same Jordan form.

The project computes:

- **Change-of-basis matrix:** given P , returns $P^{-1}AP$.
- **Similarity check:** compares eigenvalue multisets and determinants between two matrices.
- **Diagonalization:** when A is diagonalizable, displays $P^{-1}AP \approx \Lambda$ using $P =$ eigenvector matrix, confirming Strang's formula $A = S\Lambda S^{-1}$.

3.14 Inverse and Pseudoinverse

Strang reference: Ch. 2 (A^{-1}), Ch. 7.4 (A^+).

Inverse. Computed by Gauss-Jordan on the augmented matrix $[A \mid I]$: reduce A to I and the right half becomes A^{-1} . Error is thrown if $|\det(A)| < \varepsilon$ (singular matrix).

Moore-Penrose Pseudoinverse.

$$A^+ = V\Sigma^+U^T$$

where Σ^+ is formed by replacing each nonzero diagonal entry σ_i of Σ with $1/\sigma_i$. This generalizes the inverse:

- If A is square invertible: $A^+ = A^{-1}$.
- If A is tall and full column rank: $A^+ = (A^T A)^{-1} A^T$ (left inverse, gives least squares).
- If A is wide and full row rank: $A^+ = A^T (A A^T)^{-1}$ (right inverse, gives min-norm solution).
- General rank-deficient: A^+ gives the best of both.

4 Summary of Numerical Methods

The table below distinguishes theory (what Strang teaches) from the numerical algorithms used to implement that theory on a computer.

Operation	Strang theory	Numerical algorithm used
RREF, pivots, rank	Gauss-Jordan elimination	Gauss-Jordan with <i>partial pivoting</i>
LU decomposition	$A = LU$	$PA = LU$ with partial pivoting
Determinant	Product of pivots	Gaussian elim. $\times$ sign of permutation
Inverse	Gauss-Jordan on $[A \mid I]$	Same (textbook algorithm)
QR / Gram-Schmidt	Classical Gram-Schmidt	<i>Modified</i> Gram-Schmidt (more stable)
Symmetric eigenvalues	Spectral theorem	<i>Unshifted QR iteration</i>
General eigenvalues	Char. polynomial roots	<i>Faddeev-LeVerrier</i> (char. poly coeffs) + <i>Durand-Kerner</i> (all roots in $\mathbb{C}$)
SVD	$A = U \Sigma V^T$	Eigendecomp. of $A^T A$; simpler than Golub-Reinsch
Pseudoinverse	$A^+ = V \Sigma^+ U^T$	Derived from SVD
Eigenvectors	Null space of $(A - \lambda I)$	RREF on $(A - \lambda I)$
Orthonormal bases	Gram-Schmidt	Modified Gram-Schmidt

5 Convergence and Precision

- All computations use IEEE 754 double precision (≈ 15 significant decimal digits).
- The global zero tolerance is $\varepsilon = 10^{-9}$.
- QR iteration runs up to $12000 \times n$ iterations with an off-diagonal Frobenius norm convergence check. For $n \leq 5$ with well-separated eigenvalues, convergence typically occurs in < 300 iterations.
- Durand-Kerner runs up to $1000 \times n$ iterations with a max-correction convergence check of 10^{-12} .
- The SVD via $A^T A$ is reliable for matrices with condition number $\kappa(A) \lesssim 10^6$ (i.e., when squaring the matrix doesn't lose all significant digits). For ill-conditioned matrices, a proper Golub-Reinsch implementation would be required.

6 Worked Example

Consider the symmetric positive-definite matrix

$$A = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 3 & 1 \\ 2 & 1 & 3 \end{pmatrix}.$$

RREF. A has full rank 3; RREF = I_3 ; pivots at positions (0, 0), (1, 1), (2, 2); all four subspaces have their theoretical dimensions.

Determinant. $\det(A) = 4(9 - 1) - 2(6 - 2) + 2(2 - 6) = 32 - 8 - 8 = 16$.

Eigenvalues. The characteristic polynomial (from Faddeev-LeVerrier applied to the *symmetric* path, i.e., QR iteration here) gives $\lambda = 6.828\dots, 2.000, 1.172\dots$. Their sum = $\text{tr}(A) = 10$ ✓ and product = $\det(A) = 16$ ✓.

SVD singular values. $\sigma_i = \sqrt{\lambda_i(A^T A)} = \sqrt{\lambda_i(A^2)} = \lambda_i(A)$ (since A is symmetric PD), giving 6.828, 2.000, 1.172.

Solve $Ax = b$, $b = (1, 2, 3)^T$. Since A is full rank, the exact solution is $\hat{x} = A^{-1}b = (-0.75, 0.75, 1.25)^T$; residual = 0.

Jordan form. All eigenvalues distinct, all $m_a = m_g = 1$; A is diagonalizable. $P^{-1}AP = \Lambda = \text{diag}(6.828, 2.000, 1.172)$ (confirmed numerically).

7 Limitations and Future Work

1. **Complex eigenvectors:** When A has complex eigenvalues, only the eigenvalues are reported; eigenvectors require complex arithmetic. A future version could use `std::complex<double>` for the full complex Schur decomposition.
2. **Numerically rank-deficient matrices:** The rank decision (count pivots with $|\text{pivot}| > \varepsilon$) is sensitive to the choice of ε . A proper implementation would use the SVD and threshold singular values at $\varepsilon \cdot \sigma_1 \cdot \max(m, n)$.
3. **Large matrices:** All algorithms are $O(n^3)$ per iteration; QR iteration adds an $O(n)$ outer loop. For $n > 20$, specialised LAPACK routines (Householder tridiagonalization + divide-and-conquer) would be necessary.
4. **SVD accuracy for ill-conditioned matrices:** Replace $A^T A$ -based SVD with Golub-Reinsch bidiagonalization.
5. **Sparse matrices:** The current implementation stores dense $m \times n$ `double` arrays.

8 References

- [1] G. Strang, *Introduction to Linear Algebra*, 5th ed., Wellesley-Cambridge Press, 2016.
- [2] G. Strang, *Linear Algebra and Its Applications*, 4th ed., Thomson Brooks/Cole, 2006.
- [3] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013. (Reference for MGS stability analysis, QR iteration, Golub-Reinsch SVD.)

- [4] B. N. Datta, *Numerical Linear Algebra and Applications*, 2nd ed., SIAM, 2010. (Reference for Faddeev-LeVerrier algorithm and its numerical properties.)
- [5] I. O. Kerner, “Ein Gesamtschrittverfahren zur Berechnung der Nullstellen von Polynomen,” *Numerische Mathematik*, 8(3):290-294, 1966. (Original paper for the Durand-Kerner root-finding algorithm.)